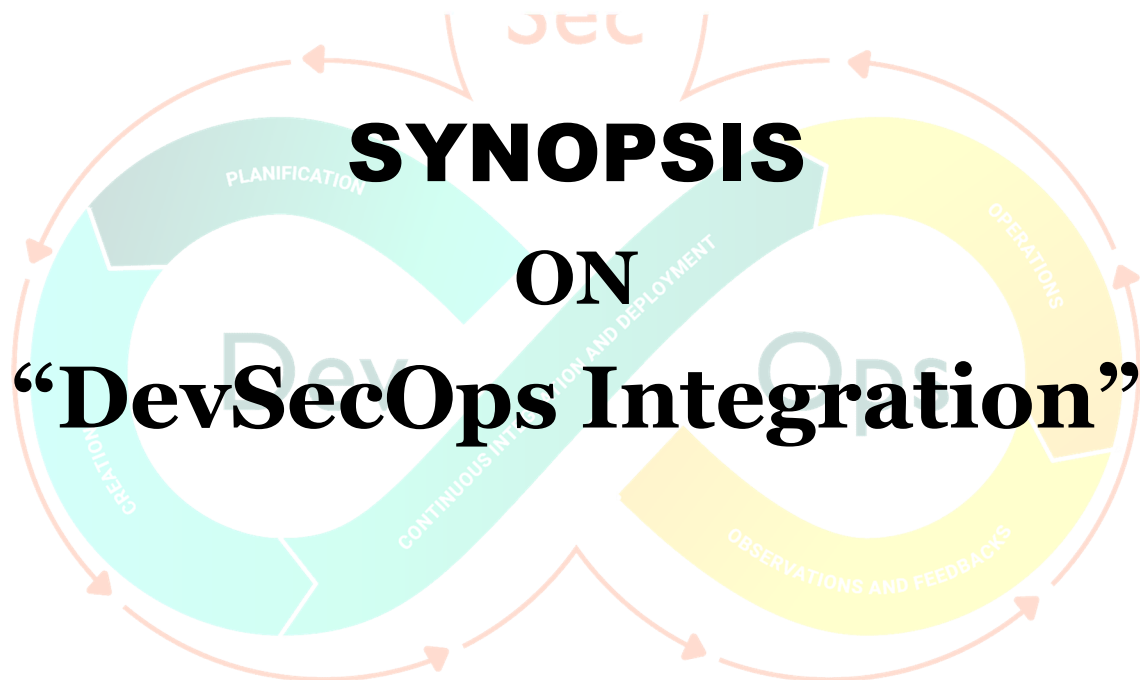




GLA
UNIVERSITY
MATHURA
Established vide U.P. Act 21 of 2010.



Submitted By Collaboration and Communication

Aditi Saraswat

2215100002

Harshita Lavania

2215100008

Submitted To

Mr. Garvit Dohere

Technical Trainer

Department of Computer Science

Objective

Implement a robust DevSecOps Pipeline for a Java-based project, integrating security practices at every stage of the software development lifecycle (SDLC) to proactively identify, mitigate, and monitor vulnerabilities while ensuring compliance and code quality. The main aim of this project is to address the growing challenges of security threats in software development by embedding security measures early in the SDLC, reducing the risk of vulnerabilities, minimizing potential breaches, and ensuring faster, more secure software delivery.

Scope

The scope of this project includes the integration of security practices into every phase of the SDLC for a Java-based application. The project covers:

- Implementing automated security checks and continuous monitoring.
- Enforcing secure coding practices and ensuring compliance with industry standards.
- Integrating tools for static and dynamic analysis, dependency scanning, and infrastructure security.
- Real-time performance and security monitoring with proactive alerts.
- Automating CI/CD pipelines with integrated security tools like Jenkins.
- Enhancing vulnerability management through continuous scanning and remediation workflows.

Boundaries and Limitations:

- The project focuses solely on Java-based applications and may not directly apply to other programming languages.
- It does not cover legacy system integration or support for non-cloud-based infrastructure.
- Manual security audits and advanced penetration testing beyond automated tools are not included.
- The project will not address end-user security training or organization-wide security policies outside the development environment.
- Third-party compliance requirements beyond standard industry best practices are out of scope.

Methodology

❖ **CI/CD – ‘Jenkins’**

Jenkins automates the building, testing, and deployment processes, ensuring continuous integration. It triggers automated builds with each code change, streamlining code quality checks and fast feedback.

❖ **Build Automation & Dependency Management – ‘Maven’**

Maven handles project dependencies and automates the build process, ensuring correct library versions. It simplifies the Java project deployment, making it easier to manage dependencies across different environments.

❖ **Static Application Security Testing – ‘SonarQube’**

SonarQube performs static analysis to detect bugs, security vulnerabilities, and code quality issues early, ensuring maintainable

and secure code. It integrates well into CI pipelines for continuous code inspection.

❖ **Dynamain Application Security Testing – ‘OWASP ZAP’**

OWASP ZAP conducts dynamic application security testing by simulating attacks on the running application, identifying vulnerabilities such as XSS and SQL injection. It helps ensure runtime security and improves application robustness.

❖ **Unit Testing – ‘JUnit’**

JUnit enables automated unit tests for Java code, ensuring individual functions work correctly. It integrates into the CI pipeline, ensuring code quality by automatically checking for bugs with every change.

❖ **Secrets Detection – ‘GitLeaks’**

GitLeaks scans Git repositories for sensitive information like passwords and API keys. It ensures no secrets are accidentally committed to the codebase, preventing potential security risks and data leaks.

❖ **Artifact Repository – ‘Nexus’**

Nexus stores build artifacts like JAR files and manages dependencies. It ensures consistency in versions, centralizes artifact management, and enhances security with access control and versioning of dependencies.

❖ **Infrastructure – ‘AWS EC2’**

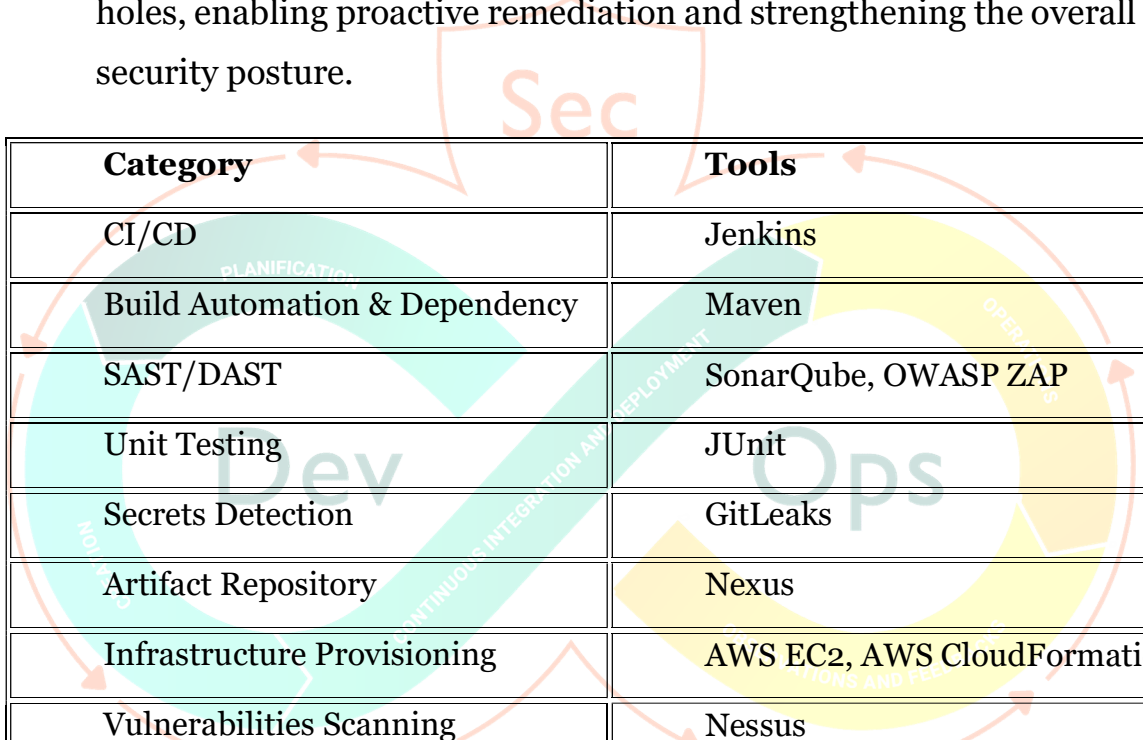
AWS EC2 provides scalable, flexible compute resources to host the application and database. It allows easy scaling and deployment of the Java application with high availability and security in a cloud environment.

❖ Infrastructure Provisioning – ‘AWS CloudFormation’

AWS CloudFormation helps automate infrastructure provisioning, making it easier to manage resources like EC2 instances, databases, and security groups.

❖ Vulnerabilities Scanning – ‘Nessus’

Nessus scans EC2 instances and infrastructure for vulnerabilities. It helps identify outdated software, misconfigurations, and security holes, enabling proactive remediation and strengthening the overall security posture.



Category	Tools
CI/CD	Jenkins
Build Automation & Dependency	Maven
SAST/DAST	SonarQube, OWASP ZAP
Unit Testing	JUnit
Secrets Detection	GitLeaks
Artifact Repository	Nexus
Infrastructure Provisioning	AWS EC2, AWS CloudFormation
Vulnerabilities Scanning	Nessus

+ Proposed System

The methodology for implementing the DevSecOps pipeline in this Java-based project follows a structured approach. This system will use industry-leading tools for Continuous Integration (CI), Continuous Deployment (CD), security testing, unit testing, dependency management, artifact management, and vulnerability scanning. The system ensures a streamlined, secure, and efficient development lifecycle, integrating the following tools:

❖ Requirement Analysis and Planning

- Identify security objectives, compliance requirements, and potential risks.

❖ Secure Development Practices

- Define and document secure coding practices in a SECURITY.md file to guide developers in writing secure code and following best practices.
- Integrate GitLeaks as a pre-commit hook to scan for hardcoded secrets in code, preventing the accidental inclusion of sensitive information like passwords and API keys.
- Perform periodic code reviews to identify and fix security vulnerabilities, ensuring adherence to secure coding practices and reducing the risk of exploits.

❖ Continuous Integration (CI) Setup

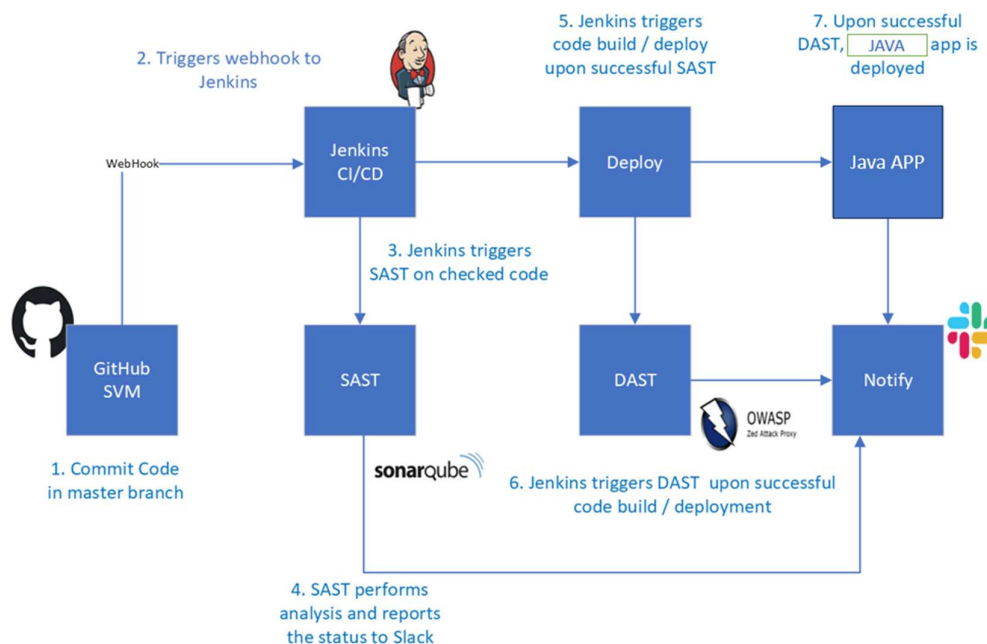
- Set up Jenkins to automatically trigger builds and tests every time a developer commits code. This ensures that the code is constantly tested and built with every change.
- Integrate SonarQube to perform static code analysis to detect bugs and vulnerabilities in the source code. Use Snyk for automated scanning of project dependencies to identify known vulnerabilities.
- Configure SonarQube with quality gates to automatically reject code with high-risk vulnerabilities or poor code quality, ensuring only secure and clean code is deployed.

❖ Automated Continuous Testing (CT)

- Set up a staging environment that mirrors production to test security measures before deployment.
- Use OWASP ZAP to perform dynamic application security testing, simulating attacks to identify runtime vulnerabilities. Postman will be used for thorough API testing to ensure secure and reliable API endpoints.

❖ Continuous Feedback and Improvement

- Set up weekly vulnerability scans with Nessus to ensure continuous security monitoring of the infrastructure and identify any emerging threats or weaknesses.
- Regularly review and update security rules, ensuring they evolve to address new vulnerabilities and emerging threats, maintaining a strong security posture.



Features

- **Automated Security Checks:** Continuous integration pipeline with automated code scanning using **SonarQube** for static code analysis and **Snyk** for dependency vulnerability scanning to identify and address vulnerabilities early.
- **Secure Code Development:** Enforced secure coding guidelines documented in the **SECURITY.md** file and integrated **GitLeaks** pre-commit hooks to detect hardcoded secrets before code submission, ensuring secure coding practices.
- **API Security:** Comprehensive API testing with **OWASP ZAP** for dynamic application security testing (DAST) and **Postman** for thorough API testing to ensure secure, well-tested endpoints.
- **Infrastructure Automation:** Automated and secure infrastructure deployment using **AWS CloudFormation** templates, ensuring that environments are consistently configured according to best security practices.
- **Continuous Improvement:** Regular vulnerability assessments and integration of security patches through weekly scans with **Nessus**, and continuous refinement of security rules based on emerging threats and vulnerabilities.

Collaboration and Communication

Implementation Plan

Week No.	Task	Mentor Evaluation
Week 1-2	security requirements, and pipeline design.	☐ Done ☐ Not Done
Week 3-4	CI Pipeline Set Up (Jenkins, SonarQube, Snyk)	☐ Done ☐ Not Done
Week 5-6	DAST/API Testing Integration and Performance Tuning	☐ Done ☐ Not Done
Week 7-8	IAC Deployment	☐ Done ☐ Not Done
Week 9-10	Weekly Nessus Scans and iterative improvements	☐ Done ☐ Not Done

Team Members

Collaboration and Communication

❖ **Harshita Lavania | Sec: 3AF | Roll No: 2215100008**

Responsible for implementing security tools, managing CI/CD pipelines, and ensuring secure coding practices.

❖ **Aditi Saraswat | Sec: 3AF | Roll No: 2215100002**

vulnerability assessment, and monitoring security compliance throughout the SDLC.

Resources Required

❖ Software & Tools:

- CI/CD: Jenkins
- Security: SonarQube, Snyk, GitLeaks, OWASP ZAP, Nessus
- Infrastructure: EC2, AWS CloudFormation
- Collaboration: GitHub

❖ Hardware & Cloud Infrastructure:

- AWS cloud resources (EC2 instances, S3, VPCs, IAM roles)
- High-performance servers for CI/CD and testing environments

References

- <https://docs.aws.amazon.com/ec2/>
- <https://www.cherryservers.com/blog/install-jenkins-ubuntu>
- <https://www.jenkins.io/doc/book/pipeline/>
- <https://www.jenkins.io/blog/2017/02/07/declarative-maven-project/>
- <https://devsecopsdocs.com/docs/introduction/getting-started>
- <https://docs.sonarsource.com/sonarqube-server/latest/>
- https://docs.gitlab.com/ee/user/application_security/dast/
- <https://junit.org/junit5/docs/current/user-guide/>
- <https://gitleaks.io/>
- <https://www.geeksforgeeks.org/top-devsecops-tools/#4-snyk>

Expected Outcomes

- **Reduced Vulnerabilities:** Early detection of security flaws in code and dependencies.
- **Improved Compliance:** Alignment with OWASP Top 10 and industry standards.
- **Faster Remediation:** Automated alerts and integration with ticketing systems.
- **Team Awareness:** Shared security ownership via accessible documentation and tools.

Conclusion

This DevSecOps pipeline ensures security is a shared responsibility across teams, enabling faster delivery of a secure, high-quality Java-based project. By automating security gates and fostering collaboration, the project minimizes risks while maintaining agility.

